

Revision Plan: Bash/OS, SSH, and Git

Use this plan to prepare for the MCQ exam. The exam checks whether you can recognize the right command or concept for common terminal, SSH, and Git situations.

1. What You Need To Know

You should be able to:

- Navigate the filesystem from the command line.
- Create, copy, and inspect files and folders.
- Understand basic file permissions.
- Generate and test SSH keys for Git hosting services.
- Initialize, inspect, and work with Git repositories.
- Stage and commit changes.
- Use remotes such as `origin`.
- Understand cloning, forking, fetching, pulling, and pushing.
- Switch branches and temporarily save work.
- Inspect Git history and file authorship.
- Remove untracked files safely.
- Undo changes using the correct Git command.
- Recognize and understand merge conflicts.

2. Bash And OS Basics

Must-Know Commands

- `pwd`: prints the current working directory.
- `ls`: lists files and folders in the current directory.
- `cd folder`: moves into a folder.
- `mkdir folder`: creates a new directory.
- `touch file.txt`: creates an empty file if it does not already exist.
- `cp source target`: copies a file.
- `mv source target`: moves or renames a file.
- `rm file`: removes a file.
- `chmod`: changes file permissions.

Key Concepts

- A working directory is the folder your terminal is currently using.
- An absolute path starts from the filesystem root, for example `/home/student/project`.
- A relative path starts from your current directory, for example `data/raw`.
- A file and a directory are different: use `touch` for files and `mkdir` for directories.
- File permissions control who can read, write, or execute a file.

Common Mistakes To Avoid

- Do not use `mkdir` when you need to create a file.
- Do not use `touch` when you need to create a directory.
- Do not confuse `cp` and `mv`: `cp` copies, while `mv` moves or renames.
- Do not assume `/data/raw` and `data/raw` mean the same thing.

Practice Tasks

- Run `pwd` and explain what the output means.
- Create a folder called `practice` using `mkdir`.
- Create a file called `notes.md` using `touch`.
- Copy `notes.md` to `backup.md` using `cp`.
- Explain when you would use `chmod`.

3. SSH Basics

Must-Know Commands And Files

- `ssh-keygen`: generates an SSH key pair.
- Public key: the key you can share with a Git hosting provider.
- Private key: the key you must keep secret.
- `ssh -T git@github.com`: tests SSH authentication with GitHub.

Key Concepts

- SSH keys are used to authenticate securely without typing your password every time.
- An SSH key pair contains a public key and a private key.
- The public key is added to services such as GitHub, GitLab, or Bitbucket.
- The private key stays on your machine and should not be shared or committed.
- Testing SSH authentication confirms that the hosting provider accepts your key.

Common Mistakes To Avoid

- Do not share your private key.
- Do not put private keys into Git repositories.
- Do not change private key permissions to unsafe values such as `777`.
- Do not assume SSH works just because the key was generated; test authentication.

Practice Tasks

- Explain the difference between a public key and a private key.
- Identify which key should be uploaded to a Git hosting service.
- Explain why `ssh -T git@github.com` is useful.

4. Git Repository Basics

Must-Know Commands

- `git init`: creates a new Git repository in the current folder.
- `git status`: shows modified, staged, and untracked files.
- `git add file`: stages changes for the next commit.
- `git commit`: records staged changes in repository history.
- `git log`: shows commit history.
- `git diff`: shows line-level changes before committing.
- `.gitignore`: tells Git which files or patterns to ignore.

Key Concepts

- A Git repository tracks changes over time.
- The working tree contains your current files.
- The staging area contains changes selected for the next commit.
- A commit is a saved snapshot of staged changes.
- Untracked files exist in the folder but are not yet tracked by Git.
- `.gitignore` is useful for generated files, temporary files, caches, and large local artifacts.

Common Mistakes To Avoid

- Do not confuse `git status` with `git log`: `status` shows current state; `log` shows history.
- Do not expect `git commit` to include files that were not staged.
- Do not use `.gitignore` to remove files that are already tracked without understanding the effect.
- Do not use `git diff` when you want to upload commits; use `git push` for that.

Practice Tasks

- Initialize a repository with `git init`.
- Create a file, then run `git status`.
- Stage the file with `git add`.
- Explain what changes after staging.
- Use `git diff` before committing and explain what it shows.

5. Remotes, Cloning, Forking, And Collaboration

Must-Know Commands

- `git clone URL`: creates a local copy of a remote repository.
- `git remote add origin URL`: links a local repository to a hosted remote.
- `git push`: sends local commits to the remote repository.
- `git fetch`: downloads remote updates without integrating them.

- `git pull`: downloads remote updates and integrates them into the current branch.

Key Concepts

- A remote is a hosted repository that your local repository can communicate with.
- `origin` is the conventional default name for a remote repository.
- Cloning creates a local copy of a remote repository on your machine.
- Forking creates a server-side copy of a repository under your own account.
- `git fetch` is safer when you want to inspect remote changes first.
- `git pull` is convenient when you want to fetch and integrate changes immediately.
- `git push` uploads your local commits to the remote.

Common Mistakes To Avoid

- Do not confuse fork and clone: a fork is hosted; a clone is local.
- Do not use `git fetch` when you mean to upload commits.
- Do not use `git push` when you mean to download remote changes.
- Do not assume a local repository can push until a remote is configured.

Practice Tasks

- Explain what `origin` usually means.
- Describe the difference between `git fetch` and `git pull`.
- Describe the difference between `git clone` and forking.
- Explain why a local repository needs a remote before pushing.

6. Branches And Temporary Work

Must-Know Commands

- `git switch branch-name`: moves to another branch.
- `git stash`: temporarily stores uncommitted changes.

Key Concepts

- Branches let you work on different lines of development.
- `git switch` is used to move between branches.
- `git stash` is useful when you have unfinished local work and need to switch context.
- Stashing helps avoid losing uncommitted work.

Common Mistakes To Avoid

- Do not use `git merge` when the task is simply to move to another branch.
- Do not use `git clean` to save work; it removes untracked files.

- Do not switch branches carelessly when you have uncommitted changes.

Practice Tasks

- Explain when to use `git switch`.
- Explain when to use `git stash`.
- Describe a safe sequence when you are on the wrong branch with unfinished work.

7. Inspecting History And Authorship

Must-Know Commands

- `git log`: views commit history.
- `git blame file`: shows which commit and author last changed each line of a file.
- `git diff`: compares current changes before committing.

Key Concepts

- Use `git log` to understand the sequence of commits.
- Use `git blame` when you need to know who last changed a specific line.
- Use `git diff` when you need to inspect current uncommitted changes.

Common Mistakes To Avoid

- Do not use `git blame` to switch branches or save work.
- Do not use `git log` to see unstaged line-level changes.
- Do not assume `git diff` shows the full project history.

Practice Tasks

- Explain what information `git log` provides.
- Explain why a teammate might use `git blame`.
- Explain how `git diff` helps before committing.

8. Cleaning, Undoing, And Conflict Handling

Must-Know Commands And Concepts

- `git clean`: removes untracked files.
- `git revert`: creates a new commit that reverses an earlier commit.
- `git commit --amend`: updates the most recent commit, often to fix its message.
- Merge conflict: happens when Git cannot automatically combine changes.
- `git cherry-pick`: applies one selected commit onto the current branch.

Key Concepts

- `git clean` is useful for unwanted untracked temporary files.
- `git revert` keeps history and adds a new commit that undoes previous changes.
- `git commit --amend` is useful for fixing the latest commit before sharing it.
- `git cherry-pick` is useful when you want one specific commit from another branch.
- A merge conflict must be resolved manually before integration can continue.

Common Mistakes To Avoid

- Do not use `git clean` if you want to keep untracked files.
- Do not use `git commit --amend` to undo an older shared commit.
- Do not confuse `git revert` with history-rewriting commands: use `git revert` when you need a history-preserving undo in shared work.
- Do not confuse a merge conflict with a detached HEAD or a fast-forward update.

Practice Tasks

- Explain when `git clean` is appropriate.
- Explain why `git revert` is safer than rewriting history in shared work.
- Explain when `git commit --amend` is useful.
- Explain what a merge conflict is in your own words.
- Explain when `git cherry-pick` is useful.

9. Mixed Scenario Practice

Use these prompts to practise choosing between related ideas rather than memorising one command at a time:

- You created a local project, added two files, and want a saved snapshot. Describe the sequence from checking the repository state to recording the commit.
- You are on the wrong branch with unfinished edits. Explain how to preserve the work, move branches, and continue safely later.
- You need updates from a teammate but want to inspect them before changing your branch. Explain which remote operation to use first and why.
- You generated logs and model artifacts during an experiment. Decide which files should be ignored, which can be deleted as untracked files, and which should be committed.
- You find a bad change in history. Explain how your choice differs if the mistake is in the latest unshared commit versus an older shared commit.

- You need one bug fix from another branch but not the whole branch. Explain the appropriate operation and how it differs from combining branches.
- You and a teammate changed the same function on different branches. Explain what Git cannot decide automatically and what a developer must do next.

10. Final Readiness Checklist

Before the exam, make sure you can solve mixed scenarios without relying on the wording of this guide:

- Given a terminal location and a target file or folder, choose whether to inspect, move, create, copy, or change permissions.
- Given two paths, explain which one is absolute and which one is relative.
- Given an SSH setup situation, identify which key is safe to share, which key must stay private, and how to confirm authentication works.
- Given a new local project, identify the steps needed to start version control and prepare changes for a commit.
- Given a `git status`-style situation, distinguish untracked, modified, staged, and committed work.
- Given a generated file, cache folder, or local artifact, decide whether it belongs in Git or should be ignored.
- Given a local repository and a hosted repository, explain how the two are connected and what `origin` usually represents.
- Given a collaboration scenario, distinguish when you are uploading local work, downloading remote work for inspection, or downloading and integrating remote work.
- Given a repository-copying scenario, distinguish between a hosted fork and a local clone.
- Given unfinished work on the wrong branch, choose a safe way to preserve work before switching context.
- Given a request to understand history, decide whether you need commit history, current line-level changes, or line authorship.
- Given unwanted temporary files, distinguish removing untracked files from undoing committed project changes.
- Given a mistake in Git history, choose between editing the latest unshared commit and creating a new commit that reverses an earlier one.
- Given a useful change on another branch, identify when applying one commit is more appropriate than integrating the whole branch.
- Given a failed integration, recognize when Git needs manual help because two branches changed the same part of a file.

11. Most Important Distinctions

- `pwd` vs `ls`: current location vs folder contents.

- `touch` vs `mkdir`: create file vs create directory.
- `cp` vs `mv`: copy vs move or rename.
- Absolute path vs relative path: starts from root vs starts from current directory.
- Public key vs private key: shared key vs secret key.
- `git status` vs `git log`: current state vs commit history.
- `git add` vs `git commit`: stage changes vs record staged changes.
- `git fetch` vs `git pull`: download only vs download and integrate.
- `git push` vs `git pull`: upload commits vs bring in remote changes.
- Clone vs fork: local copy vs hosted copy under your account.
- `git stash` vs `git clean`: save uncommitted work vs remove untracked files.
- `git revert` vs `git commit --amend`: undo an earlier commit with a new commit vs edit the most recent commit.
- `git switch` vs `git merge`: move branches vs combine branches.
- `git cherry-pick` vs `git merge`: apply one commit vs integrate a branch.